Rollcall

Rollcall

Face-recognition attendance for real classrooms

Project report

November 2026

rollcall-classroom.vercel.app

github.com/LSUDOKO/ai_project

Contents

1 Summary 3

2 The problem and the approach 4
2.1 The problem 4
2.2 Where this started 4
2.3 The approach 4

3 How it works for each person 5
3.1 The teacher 5
3.2 The student 7

4 System architecture 9
4.1 A live session, message by message 9

5 Face recognition pipeline 11
5.1 Why YuNet and SFace replaced DeepFace 11
5.2 Enrolment 11
5.3 Recognition in a session 12
5.4 Choosing the threshold 13

6 Data model 14
6.1 Rules worth knowing 14

7 API summary 15

8 Security and privacy 16
8.1 Who can see what 16

9 Testing and results 17
9.1 API tests 17
9.2 Browser flow 17
9.3 Recognition accuracy 18
9.4 Container performance 18

10 Deployment and operations 19
10.1 Environment variables 19
10.2 Living with the free tier 19

11 Known limits and next steps 20

Appendix: local setup 21

1 Summary

Rollcall takes attendance by face. A teacher opens a session and points a camera at the room, and the register fills itself. Each student enrols once, on their own phone, with consent.

A teacher creates a class and gets a six-letter code. Students sign up, join with that code, and take five photos. Those photos become a face template of 128 numbers, and the photos are discarded. During a session, the browser sends camera frames to the API, which matches faces against that class's students only and writes attendance rows. The teacher can correct any entry by hand, even after the session ends, and export the register as a CSV that opens in Excel. Students see only their own record.

The product grew out of an old desktop app built on Tkinter and OpenCV, which is kept in `legacy/`. Rollcall is a rewrite for the situation that app could not serve: many students, each with their own device, and a teacher who has a few minutes at the start of a class.

Live site	https://rollcall-classroom.vercel.app
Frontend	Next.js 16 on Vercel
API	Flask in Docker on Render's free tier, Singapore
Database	Postgres on Neon, Singapore
Face models	OpenCV YuNet detector and SFace recogniser, compared by cosine similarity
Accuracy	On the LFW 6,000-pair benchmark at threshold 0.42: false accept 0.0% (0 of 3,000), false reject 3.4%. These are single-photo pair results on a public benchmark, not a classroom study.
Tests	9 API tests against real Postgres and the real models; a browser flow with 24 checkpoints using fake camera feeds

Rollcall has clear limits, set out in section 11. The most important is that it has no liveness detection: a good photo may fool it, so teachers should keep an eye on the camera. It has been tested with synthetic faces, not in a real classroom.

2 The problem and the approach

2.1 The problem

Calling names uses up the start of every class, and a paper register is easy to sign for someone else. It then has to be copied somewhere before anyone can work out who is falling behind. Face recognition can remove the calling and the copying, but only if students accept it and teachers can trust the result. A system that keeps photos of students, or marks people the teacher cannot overrule, solves one problem by creating a worse one.

2.2 Where this started

The first version was a desktop app. One machine ran a Tkinter window, captured up to 50 photos per student into a folder, trained an LBPH model, and wrote a CSV per subject. It proved the idea but needed each student to stand in front of that one machine. A later web version in the git history moved to Flask, MongoDB, MTCNN and DeepFace. Rollcall replaced that backend and its schema. Table 1 shows the three.

Table 1 Three generations of the project.

	Desktop app	Web prototype	Rollcall
Where	`legacy/`	Git history, before commit 2930098	Current code
Interface	Tkinter window	Next.js and Flask	Next.js and Flask
Detection	Haar cascade	MTCNN	YuNet
Recognition	LBPH, trained per install	DeepFace, Facenet512	SFace, no training step
Storage	Image folder and CSV files	MongoDB	Postgres; templates, no photos

2.3 The approach

- **Enrol once, on your own phone.** Enrolment happens away from class time. A student's face works in every class they join.
- **Keep numbers, not photos.** Frames are decoded in memory for one request. Only a numeric template is stored, and only after explicit consent.
- **Match against the class, not the world.** A session compares faces with that class's enrolled students only, so a visitor cannot be marked present.
- **The teacher has the last word.** Every entry can be changed by hand, and the register shows whether each mark came from the camera or from a person.
- **Refuse when unsure.** A weak match, or a face that is too close to two students, is reported as not recognised rather than guessed.
- **Fit the free tier.** The whole system runs on free hosting, which shaped the choice of models (section 5.1) and the handling of cold starts (section 10.2).

3 How it works for each person

The screenshots in this section come from the automated browser flow described in section 9.2. All faces are AI-generated synthetic images, not real people, and the names are invented test data.

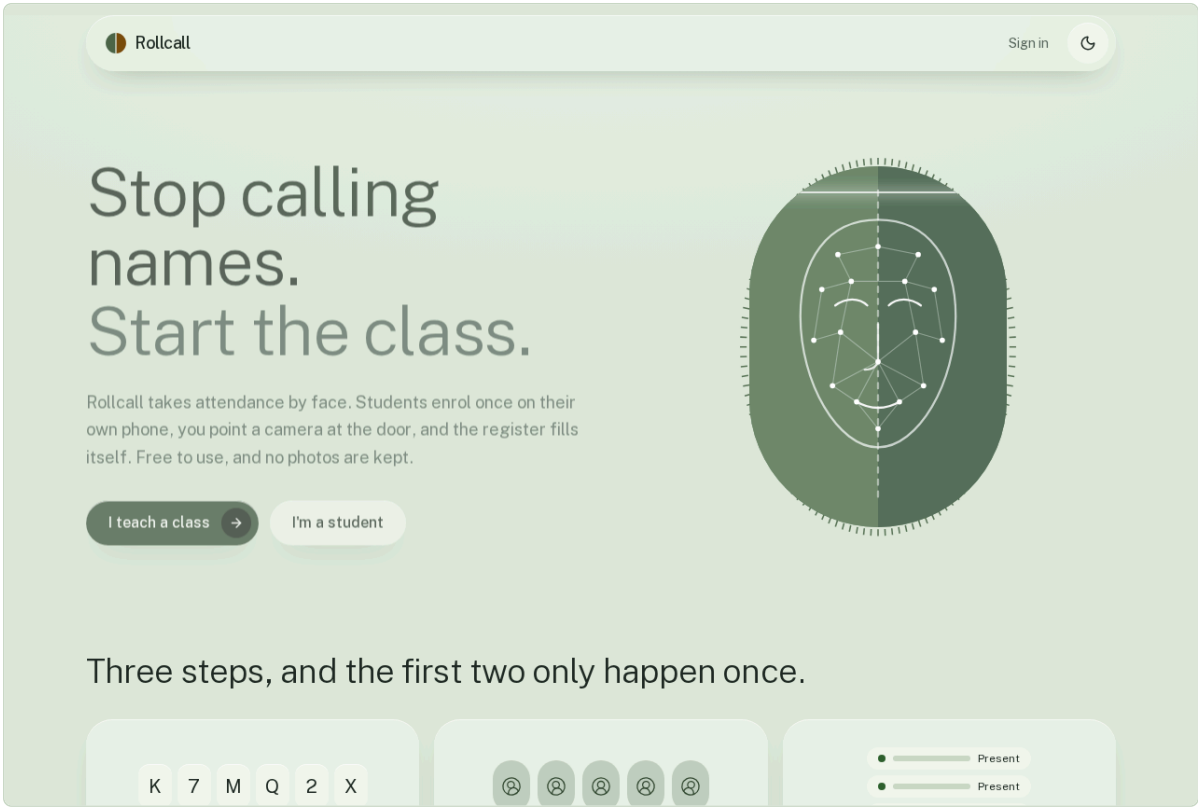


Figure 1 The landing page, which leads with a choice between teaching and being a student. It also pings the API on load, so a sleeping server starts waking before anyone signs in.

3.1 The teacher

Create a class. After signing up as a teacher, the home page lists classes and recent sessions. Naming a class generates a six-letter code from an alphabet that leaves out easily confused characters such as 0, O, 1, I and L, because codes get read aloud. The teacher shares the code or an invite link.

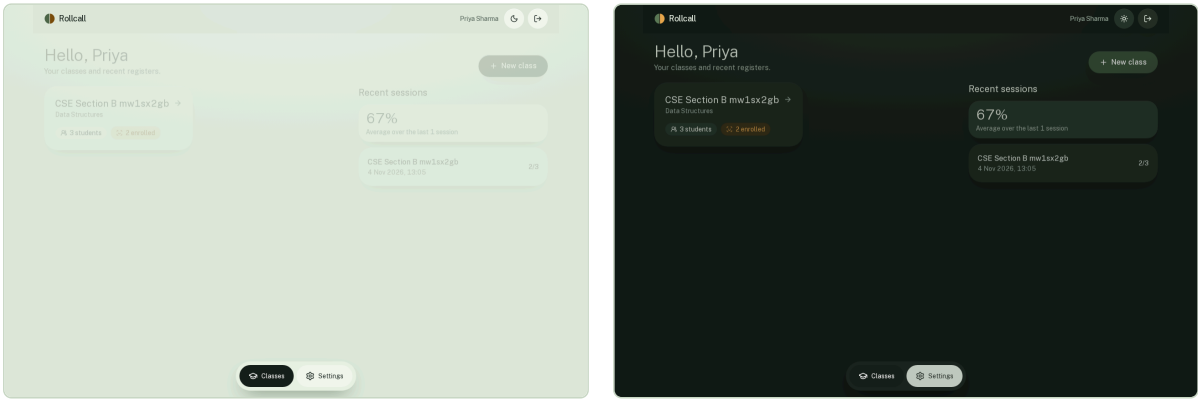


Figure 2 The teacher's home page in the light and dark themes.

Watch the roster fill. The class page shows who has joined and who has enrolled a face. Students without a face can still be marked by hand.

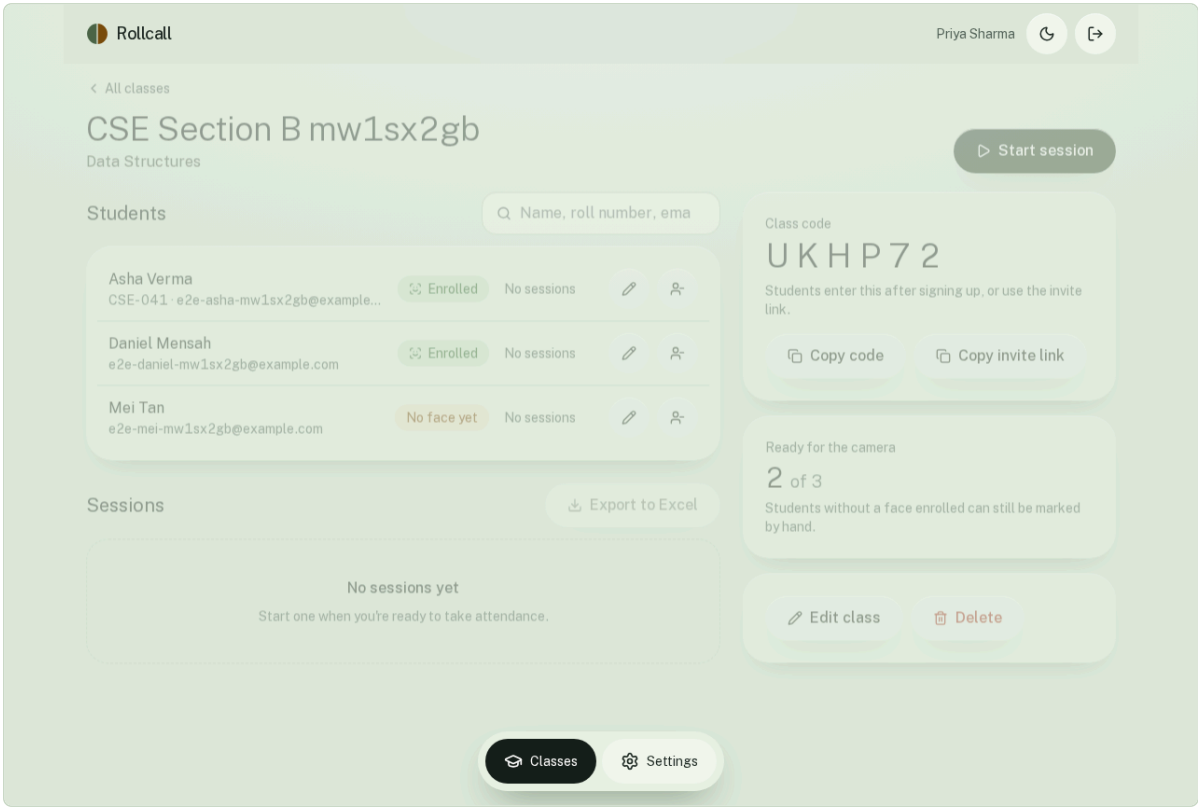


Figure 3 The class page: roster with enrolment status, class code, invite link and session history.

Run a session. The teacher starts a session, optionally names the topic, and starts scanning on a laptop or phone. Each face gets a box on the video: green with a name when it is matched, amber with "Not recognised" when it is not. The register beside the video updates when someone new is marked.

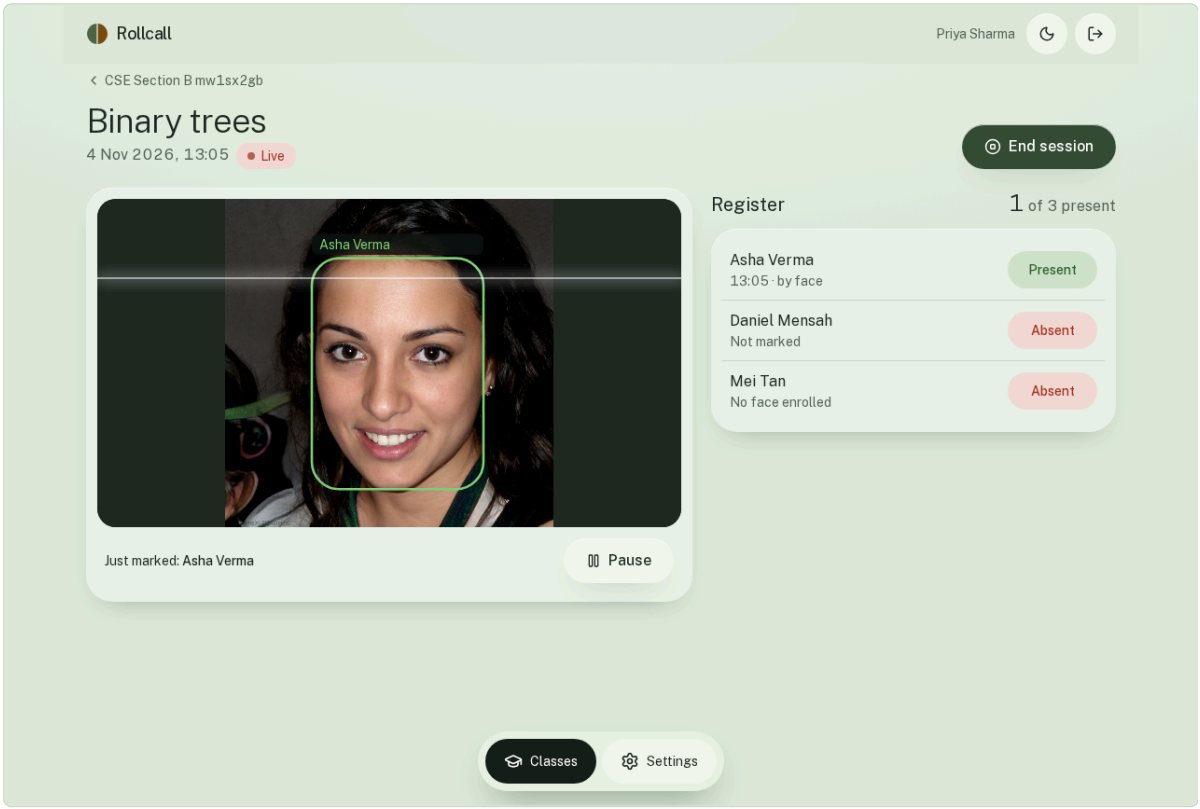


Figure 4 A live session. The camera has matched one student and recorded the time and method. The face is synthetic.

Correct and finish. Tapping a student's status switches it between present and absent, recorded as a manual mark. Ending the session shows a summary. The register can still be corrected afterwards, and it exports as a CSV with the name, roll number, email, status, time and method. The class page also exports the full register, with every student against every finished session.

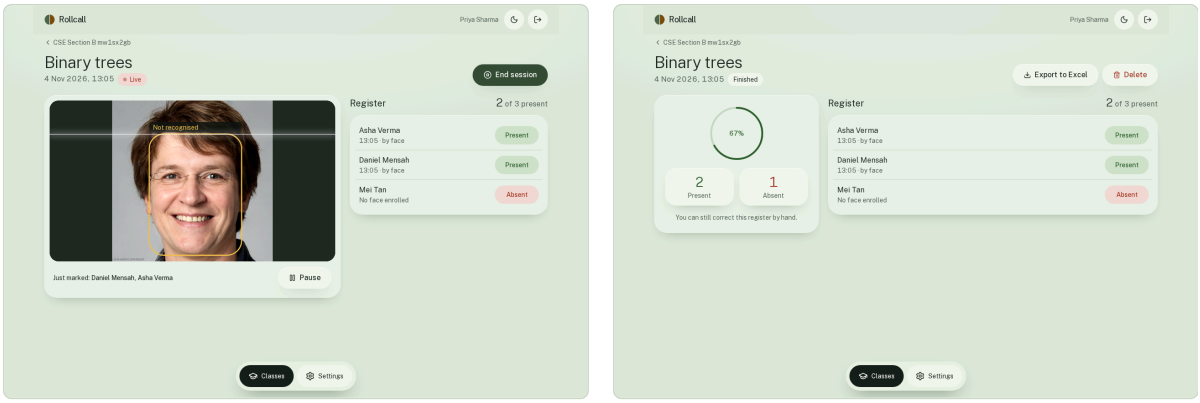


Figure 5 Left: a face outside the class stays "Not recognised" and marks nobody. Right: the finished session, with two of three present.

3.2 The student

Sign up and join. An invite link joins the class automatically after sign-up. Otherwise the student types the code. The home page shows a short checklist: add a roll number, enrol a face, join a class.

Enrol. The camera button stays disabled until the consent box is ticked. The page then asks for five photos, looking straight ahead, left, right, up and down. Each photo is checked straight away, so a bad one is caught when it is taken. A student can then test recognition before class. The test tells them whether the camera knows them, and nothing goes on their attendance.

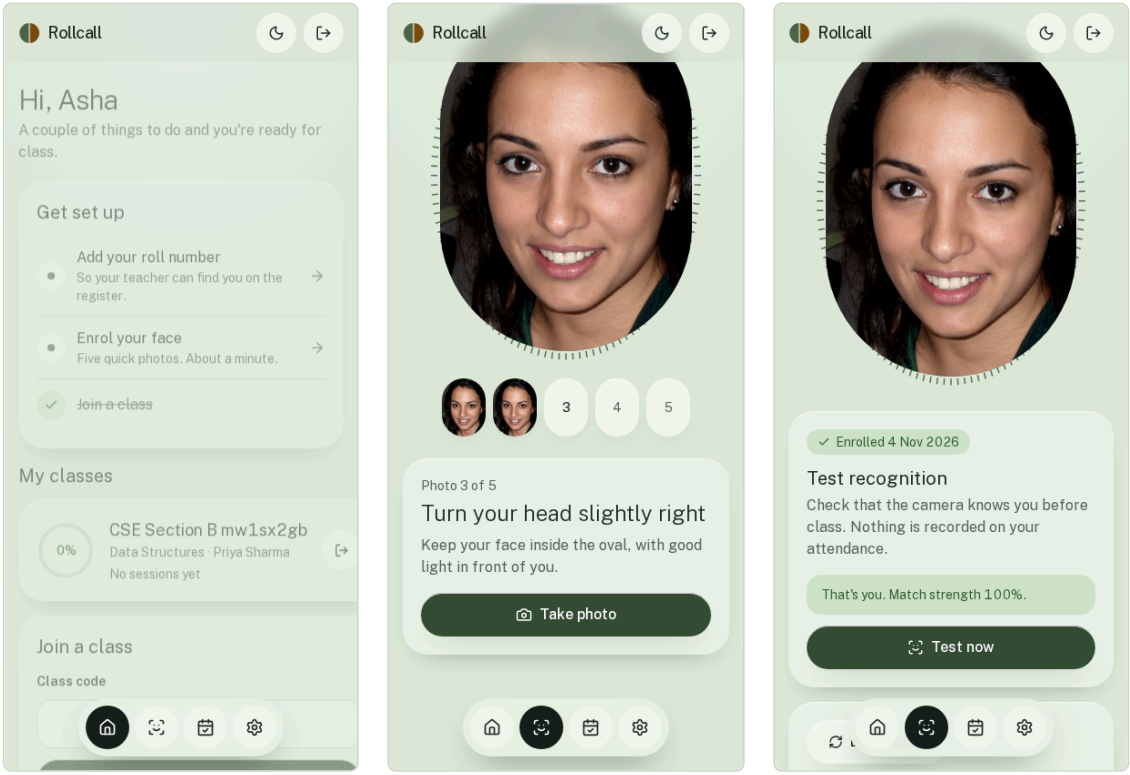


Figure 6 On a phone: the student's home page, enrolment at photo 3 of 5, and the recognition test. The faces are synthetic.

See your own record. The attendance page lists finished sessions since the student joined the class, with present and absent counts and a rate. It filters by class and date, and exports to a CSV. A session appears only after the teacher ends it. Students have no way to see anyone else's attendance.

Stay in control. From the My face page and from settings, a student can delete their template or their whole account at any time. Enrolment is optional, and a teacher can always mark a student by hand instead.

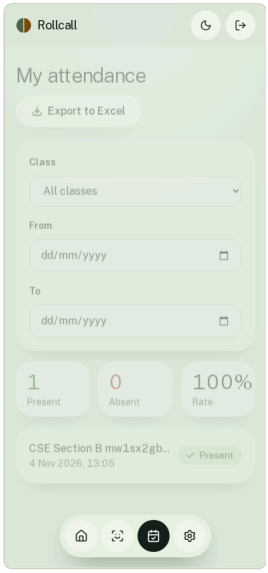


Figure 7 A student's own attendance.

4 System architecture

Rollcall has three deployed parts. Vercel serves the web app. The browser then calls the API on Render directly, across origins, so no attendance data or camera frame passes through Vercel. The API is the only part that talks to the database, and the only place face templates are read.

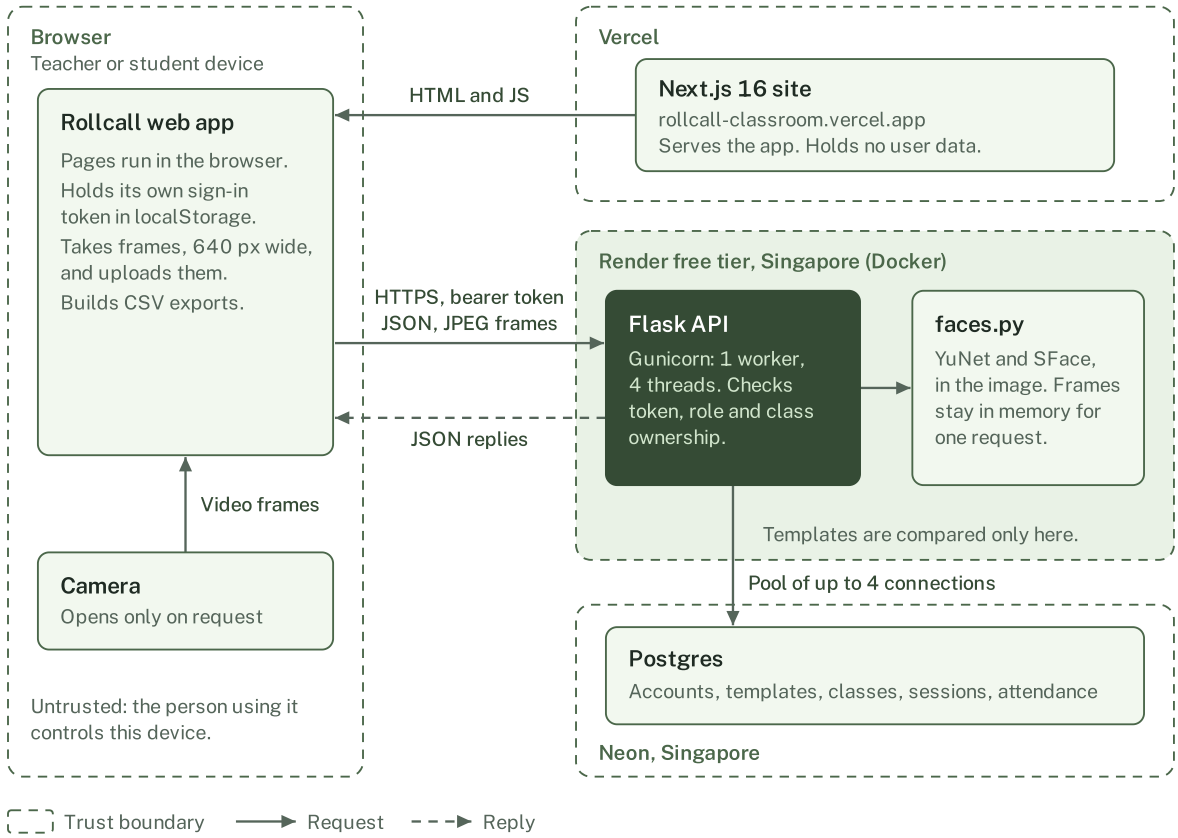


Figure 8 System architecture and trust boundaries. Face templates are written and read inside the Render and Neon boundaries, and no API response includes them.

Table 2 Components.

Component	Role
`frontend/`	Next.js 16 and React 19 with Tailwind CSS. Pages are client components that call the API through `lib/api.ts`.
`backend/app.py`	All 28 routes: accounts, enrolment, classes, sessions, recognition and reports. Applies `schema.sql` on every boot, so each statement is idempotent.
`backend/faces.py`	Image decoding, detection, embedding and matching. Inference runs behind one lock because the OpenCV models hold per-call state.
`backend/schema.sql`	Five tables, their indexes, and the rule that a class has one open session at a time.

4.1 A live session, message by message

A session is an ordinary database row. Starting one twice returns the same open session, enforced by a partial unique index. Scanning is a loop in the browser: send a frame, wait for the reply, wait 700 ms, repeat. A slow server therefore slows the loop down rather than building up a queue of requests. Figure 9 shows the messages.

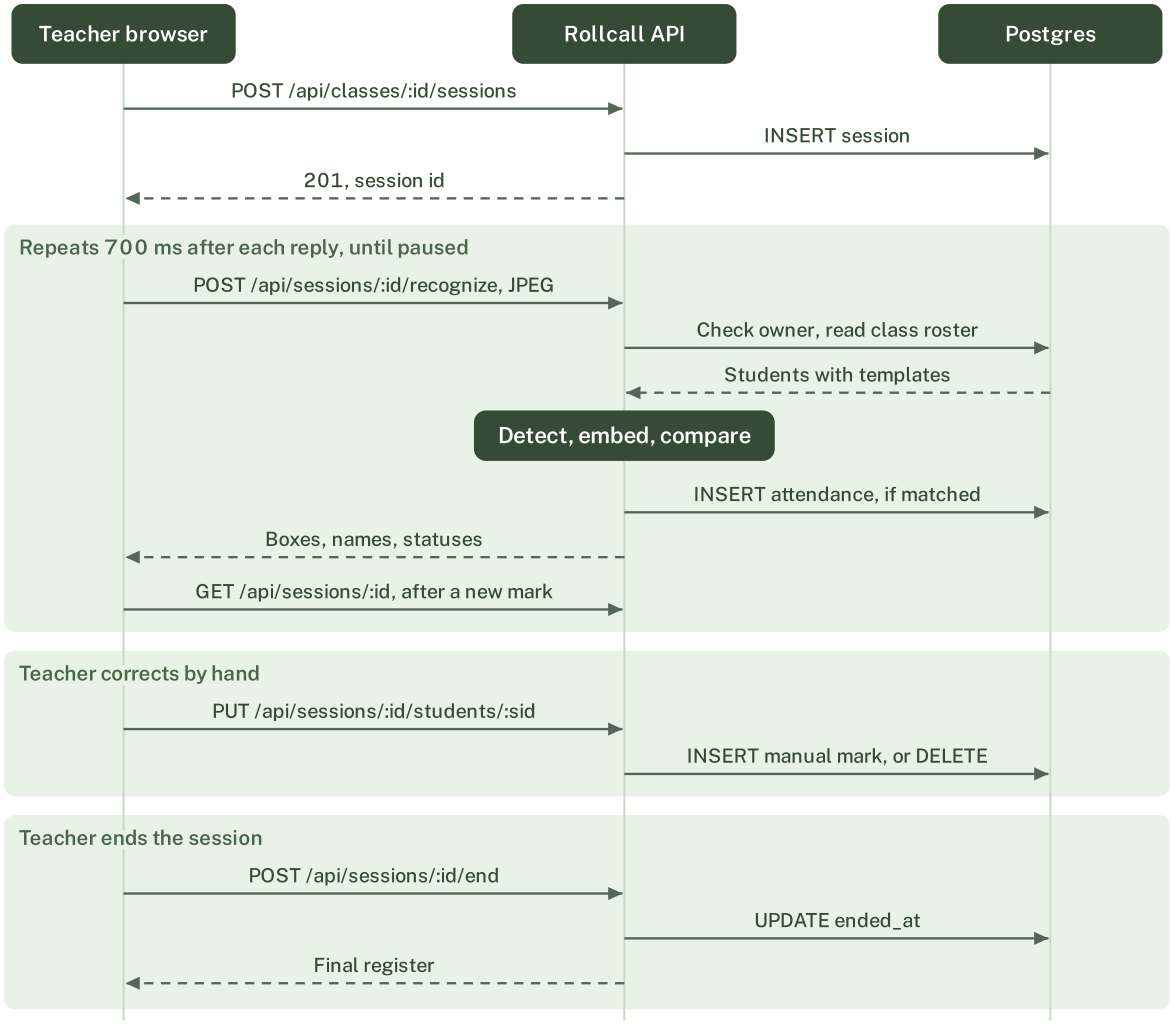


Figure 9 A live session. Solid arrows are requests and dashed arrows are replies. Manual corrections also work after the session has ended; recognition does not, and returns 409.

5 Face recognition pipeline

5.1 Why YuNet and SFace replaced DeepFace

The previous backend detected faces with MTCNN and computed embeddings with DeepFace's Facenet512 model, which brings TensorFlow with it. That stack does not fit a 512 MB instance, and the free tier provides exactly that. Rollcall now needs only `opencv-python-headless` and two ONNX models from the OpenCV Zoo: YuNet, about 230 KB, and SFace, about 37 MB. The Dockerfile bakes both into the image and pins them by SHA-256, so a cold start never downloads anything. Container measurements are in section 9.4.

This was a choice driven by memory. Facenet512 and SFace were not compared on the same benchmark, so this report makes no claim that one is more accurate than the other.

A template is 128 numbers scaled to unit length, so cosine similarity is a dot product. The values in Table 3 are constants in `faces.py` and `app.py`.

Table 3 Recognition settings in the code.

Setting	Value	Meaning
`MAX_SIDE`	640 px	Longest side of an image after decoding
`DETECTION_SCORE`	0.8	YuNet confidence required to report a face
`MIN_FACE_PX`	60 px	Smaller faces are ignored
`MATCH_THRESHOLD`	0.42	Similarity needed to count as the same person; set by `FACE_MATCH_THRESHOLD`
`MATCH_MARGIN`	0.05	How far the best match must beat the runner-up
Duplicate check	0.52	Threshold plus 0.1: a template this close to another account is refused
`ENROLL_SHOTS`	5	Photos required for enrolment

5.2 Enrolment

Enrolment has to catch two kinds of mistake: bad photos, and photos of the wrong person. The second check on the server handles mixed photos. Each of the five embeddings is compared with the average of the *other four*. Comparing it with an average of all five would let a different person through, because they would partly be matching themselves.

The third check enforces one person, one account. The new template is compared with every enrolled user in the database, not only those in one class. This stops a friend's face being enrolled on your account so that they can sign in for you. The stricter 0.52 threshold keeps two similar-looking students from blocking each other.

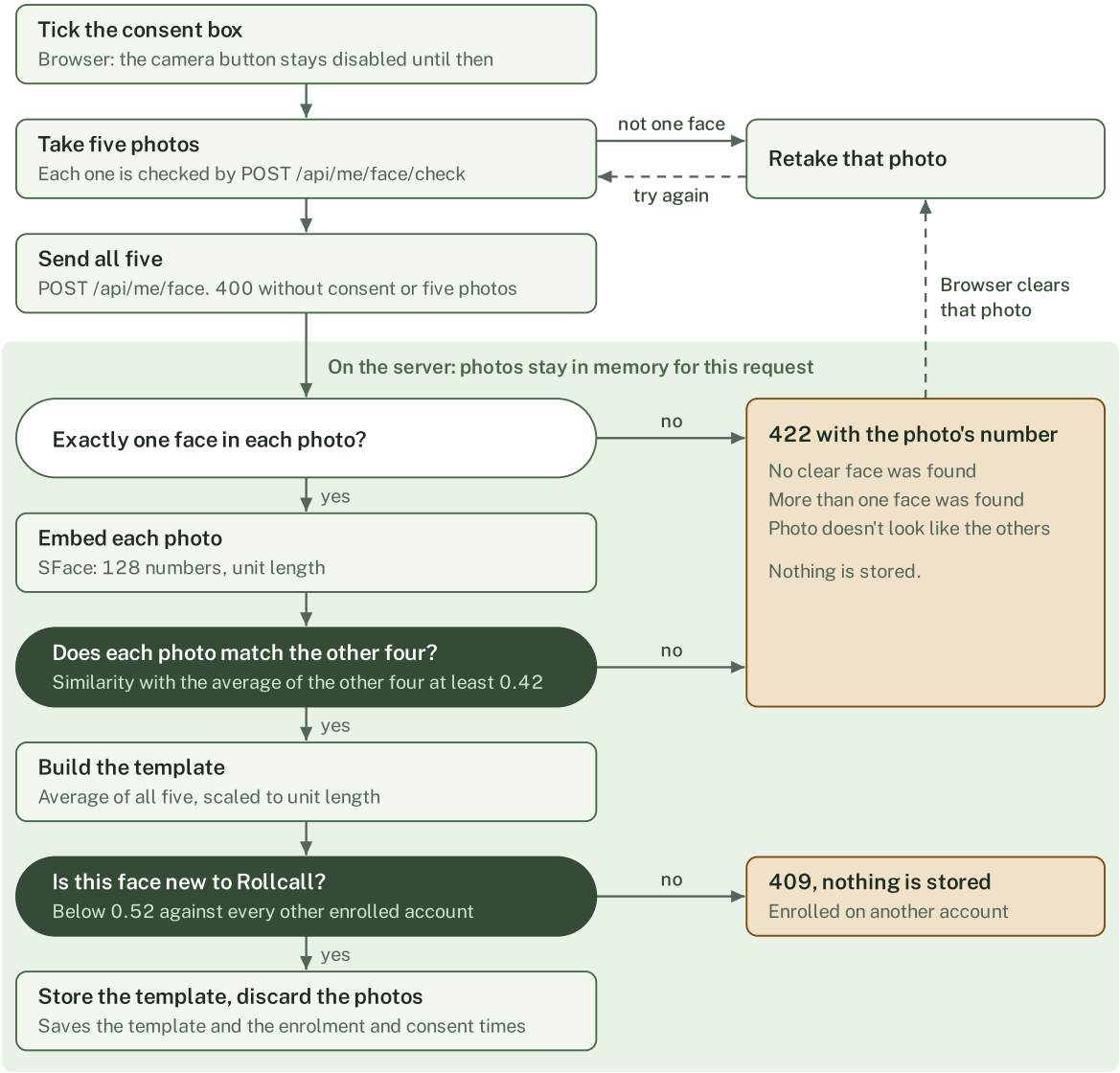


Figure 10 Enrolment. The two dark steps are the checks that stop a wrong or second face getting on an account.

5.3 Recognition in a session

Figure 11 follows one frame from the camera to an attendance row. The exactly-one-face rule belongs to enrolment and the self-test. A live session deliberately handles every face in the frame, because a teacher may have several students in view. It has its own guards instead: faces are handled biggest first, one student can be claimed only once per frame, and the database primary key allows one attendance row per student per session. If a repeat match reaches the insert, it reports "already" and changes nothing.

With one enrolled student in a class there is no runner-up, so only the threshold applies. A student who has not enrolled cannot match, and can only be marked by hand.

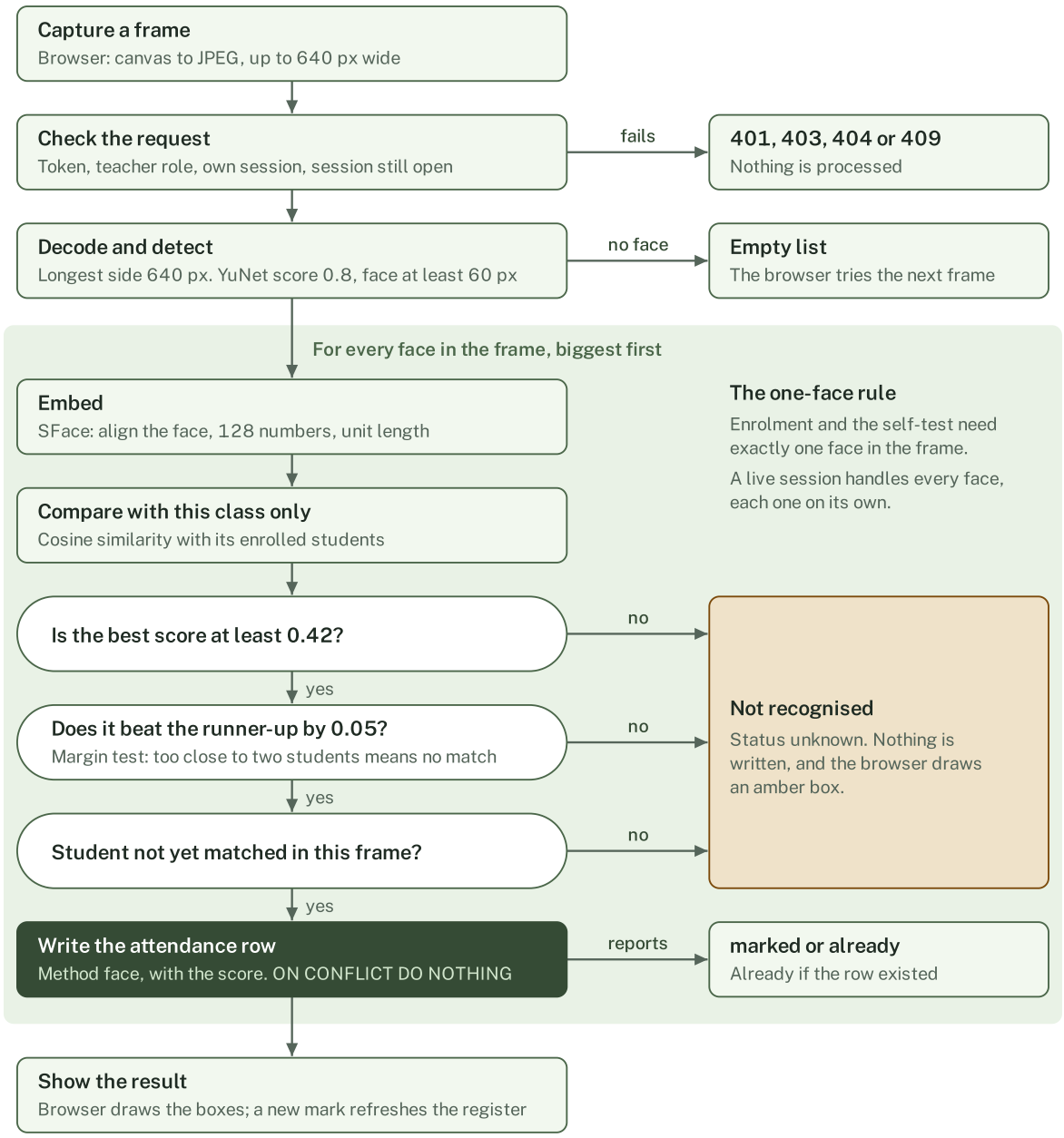


Figure 11 From camera frame to attendance row. A face that fails any of the three tests marks nobody.

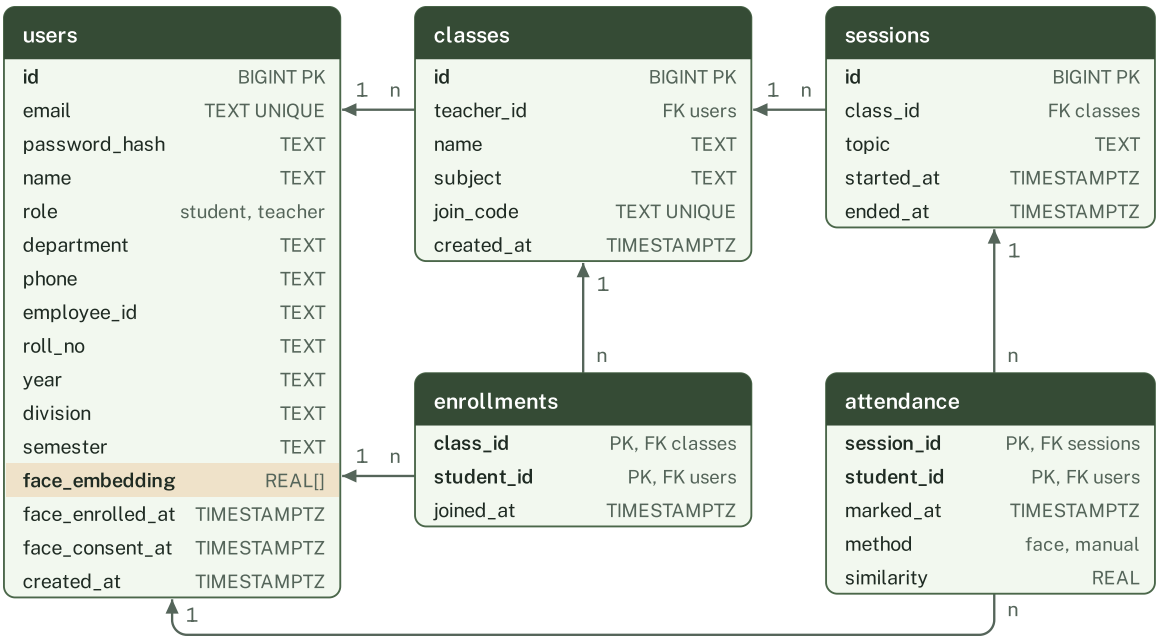
5.4 Choosing the threshold

`backend/tests/evaluate_threshold.py` runs the real detector and recogniser over labelled face pairs and prints false accept and false reject rates for thresholds from 0.30 to 0.60. On the LFW standard 6,000 pairs, 0.42 was chosen: no different-person pair reached it, and the highest different-person score was 0.412. False rejects at that point were 3.4%.

In a register, a false accept marks an absent student present with no visible sign, while a false reject costs a retry on the next frame or a tap by the teacher. That is why the threshold sits just above the highest different-person score. The gap is narrow, only 0.008, so the result should not be read as a guarantee. The margin test and class-only matching add protection that the benchmark does not measure. Full results are in section 9.3.

6 Data model

Five tables hold everything. All foreign keys cascade on delete, which is what makes account deletion complete: removing a user removes their classes, enrolments and attendance. The schema has no table for photos, because no photos are kept.



Arrows run from a foreign key to the table it references. The highlighted column is the biometric template.

Figure 12 Entity relationships, from `backend/schema.sql`.

6.1 Rules worth knowing

- **A row means present.** The `attendance` table has no absent rows. A student is absent from a session if they had joined by then and have no row. Removing a mark therefore deletes the row.
- **Late joiners are not penalised.** Totals count only sessions that started after the student's `joined_at`.
- **One open session per class.** The partial unique index `one_open_session` covers `class_id` where `ended_at` is null, so two teachers' tabs cannot open rival sessions.
- **One row per student per session.** The primary key, with `ON CONFLICT DO NOTHING`, makes repeated recognition harmless. The first mark's time and method are kept.
- **Consent is recorded.** `face_consent_at` is set with the template and cleared with it.
- **Emails are lowercase.** A check constraint enforces it, and the API lowercases on the way in.

7 API summary

Requests and responses are JSON. Authenticated routes need an `Authorization: Bearer` header. The Role column shows who may call each route: another role receives 403. A teacher who asks for another teacher's class or session receives 404, so the API does not reveal that it exists.

Table 4 All 28 routes in `backend/app.py`.

Method	Path	Role	Purpose
Service and accounts			
GET	`/health`	Public	Health check, also used to wake the server
POST	`/api/signup`	Public	Create an account and return a token
POST	`/api/signin`	Public	Sign in and return a token
GET	`/api/me`	Any	Read own profile
PATCH	`/api/me`	Any	Update own profile
POST	`/api/me/password`	Any	Change password, signing out other devices
DELETE	`/api/me`	Any	Delete the account, after confirming the password
Student			
POST	`/api/me/face/check`	Student	Count faces in a frame; once enrolled, test the match
POST	`/api/me/face`	Student	Enrol from five photos, with consent
DELETE	`/api/me/face`	Student	Delete the template and consent record
GET	`/api/student/classes`	Student	Own classes with attendance totals
POST	`/api/student/classes/join`	Student	Join a class by code
DELETE	`/api/student/classes/:id`	Student	Leave a class
GET	`/api/student/attendance`	Student	Own history, with class and date filters
Teacher: classes			
GET	`/api/classes`	Teacher	Own classes and recent sessions
POST	`/api/classes`	Teacher	Create a class and its join code
GET	`/api/classes/:id`	Teacher	Class, roster and sessions
PATCH	`/api/classes/:id`	Teacher	Rename a class or change its subject
DELETE	`/api/classes/:id`	Teacher	Delete a class and its registers
GET	`/api/classes/:id/report`	Teacher	Students by sessions, for the export
PATCH	`/api/classes/:id/students/:sid`	Teacher	Edit a student in this class
DELETE	`/api/classes/:id/students/:sid`	Teacher	Remove a student from this class
POST	`/api/classes/:id/sessions`	Teacher	Start a session, or return the one already open
Teacher: sessions			
GET	`/api/sessions/:id`	Teacher	Session with its register
POST	`/api/sessions/:id/recognize`	Teacher	Match the faces in one frame and mark attendance
PUT	`/api/sessions/:id/students/:sid`	Teacher	Set present or absent by hand, even after the end
POST	`/api/sessions/:id/end`	Teacher	End the session and return the register
DELETE	`/api/sessions/:id`	Teacher	Delete a session and its attendance

8 Security and privacy

Face data is sensitive, and attendance is a record people may rely on. Table 5 lists each control and its limit.

Table 5 Security and privacy controls.

Control	What the code does	Limit
Tokens	Signed with `itsdangerous`, valid for 14 days. Each carries the last 12 characters of the password hash, so changing a password signs out every other device.	Kept in `localStorage`, which a script running on the site could read.
Passwords	Hashed with Werkzeug's salted scrypt. Deleting an account needs the password again.	Email addresses are not verified.
Role checks	Every protected route declares its role, checked against the database on each request. Students cannot reach teacher routes or mark themselves present.	Anyone can sign up as a teacher, which only gives access to classes they create.
Class scoping	Every class and session lookup includes the teacher's id. Another teacher's data returns 404. Recognition compares only the class's enrolled students.	
Embeddings stay on the server	The column is left out of the public column lists and is read only to compare faces. A test asserts that no response contains "embedding" or "password".	Stored as plain numbers in Postgres. The app adds no column encryption.
Photos	Frames are decoded in memory and never written to disk or the database. Enrolment previews stay on the student's device until they are uploaded.	Frames travel to the API as data, over HTTPS.
Consent	The API requires `consent` to be exactly `true` and records when it was given. The camera button waits for the tick box.	
Deletion	A student can delete the template at once, or the whole account, which cascades to attendance. A teacher's deletion also removes their classes and registers.	Backups held by the database host were not examined.
CORS	Only the production site and origins in `FRONTEND_ORIGIN` are allowed, and only the two headers the app needs.	CORS limits browsers. It is not access control; the token is.
Rate limits	See Table 6. Attempts over the limit receive 429.	In memory, per process, and reset on restart.
CSV exports	A cell starting with `=`, `+`, `-`, `@`, a tab or a carriage return gets a leading apostrophe, so a hostile name cannot run as a formula in Excel. Quotes are escaped.	
Input limits	Requests are capped at 12 MB, and unexpected errors return a generic message.	

Table 6 Rate limits. This is basic abuse protection and nothing stronger.

Action	Limit	Counted by	Note
Sign up	20 per hour	Forwarded address	The header can be set by the client, so this can be evaded
Sign in	10 per 5 minutes	Email	Someone else's failed attempts can lock an account for 5 minutes
Change password, delete account	10 per 5 minutes	User	Shared counter
Join a class	20 per 5 minutes	User	Slows guessing of class codes

8.1 Who can see what

A teacher sees the students who joined their class: name, email, profile details, and attendance in that class, including the match score for camera marks. A student sees only their own record. Nobody can see a face template through the app, including the student it belongs to. The template is numbers rather than an image, but it is still biometric data and is treated as sensitive.

9 Testing and results

9.1 API tests

`backend/tests/test_api.py` holds 9 tests. They run against a real Postgres and the real face models, with nothing mocked.

Table 7 The API tests.

Test	What it establishes
`signup_validates_and_rejects_duplicates`	Bad email, short password and unknown role are refused; a duplicate email gets 409, ignoring case
`signin_and_token_rules`	Wrong password and forged token fail; old tokens stop working after a password change
`signin_is_rate_limited`	The eleventh attempt gets 429
`cors_only_allows_the_site`	An allowed origin gets the header and another origin does not
`full_attendance_flow`	Join, enrol, consent, mixed-photo refusal, duplicate refusal, recognition, strangers, ending, manual correction, reports and filters
`data_is_scoped_to_its_owner`	Another teacher gets 404 on eight routes; a student gets 403 on teacher routes
`only_class_members_can_match`	An enrolled student outside the class stays unknown
`no_embedding_ever_leaves_the_server`	Four responses contain neither "embedding" nor "password"
`bad_images_and_delete_account`	Bad images get 400; account deletion needs the password and takes effect

9.2 Browser flow

`frontend/e2e/flow.mjs` drives the whole product through four real Chromium browsers, one per person, each with a fake camera feed of synthetic faces: a teacher on a desktop viewport and three students on phone viewports. It passes 24 checkpoints and fails if any browser logs a console error. Along the way it checks that:

- the invite link joins a class automatically, and a wrong code gets a clear message;
- the camera button waits for consent;
- a student is redirected away from teacher pages;
- the camera marks two enrolled students, leaves a stranger unrecognised, and does not mark the student with no template;
- manual corrections work, and the session ends with 2 of 3 present;
- all three CSV exports contain the right rows;
- each student sees the correct result;
- password change, account deletion and face data deletion work.

9.3 Recognition accuracy

Accuracy was measured with `backend/tests/evaluate_threshold.py` on the Labeled Faces in the Wild (LFW) standard 6,000 pairs. One pair was skipped.

Table 8 LFW results at threshold 0.42.

Measure	Result
Same-person pairs evaluated	2,999
Different-person pairs evaluated	3,000
False accept rate	0.0% (0 of 3,000)
False reject rate	3.4%
Highest different-person score	0.412

> **How to read these numbers.** They are single-photo pair results on a public benchmark, not a classroom study. The app enrols five photos, and classroom light, angles and cameras will differ from LFW, so accuracy in a real room has not been measured. Zero false accepts out of 3,000 does not mean false accepts cannot happen.

9.4 Container performance

Table 9 Container performance. Measurements pending.

Measure	Result
Memory at idle	Pending
Peak memory during recognition	Pending
Recognition request time	Pending
Cold start time	Pending

10 Deployment and operations

Table 10 Where each part runs.

Part	Host	Details
Frontend	Vercel	`https://rollcall-classroom.vercel.app`
API	Render free tier, Singapore	`https://rollcall-api-w318.onrender.com`. Docker image on Python 3.11, defined by `render.yaml`, with health checks on `/health`. Gunicorn runs 1 worker with 4 threads.
Database	Neon, Singapore	Postgres. The API applies `schema.sql` at each boot.

10.1 Environment variables

Table 11 Configuration. Secrets are set in each host's dashboard and are not in the repository.

Variable	Where	Purpose
`DATABASE_URL`	API	Postgres connection string. Required.
`SECRET_KEY`	API	Signs tokens. Required. Generated by the Render blueprint. Changing it signs everyone out.
`FRONTEND_ORIGIN`	API	Extra allowed sites, comma-separated. The production site is always allowed.
`FACE_MATCH_THRESHOLD`	API	Optional. Defaults to 0.42.
`MODEL_DIR`	API	Optional. Defaults to `models/` beside `faces.py`.
`PORT`	API	Set to 10000 in the Dockerfile; 5000 when running `app.py` directly.
`NEXT_PUBLIC_API_URL`	Frontend	Base URL of the API, fixed at build time.

10.2 Living with the free tier

- **Sleeping server.** Render's free tier sleeps when idle, so the first request afterwards can take about a minute. If a request takes longer than 3.5 seconds, the site shows a banner saying the server is waking up. The landing page calls `/health` on load, so the wake-up often starts before anyone signs in. Scan requests do not trigger the banner, and a failed scan shows an error while the loop keeps trying.
- **Sleeping database.** Neon closes connections when its compute scales to zero. The connection pool checks each connection before use, with a pool of one to four connections.
- **512 MB of memory.** This decided the choice of models (section 5.1). It is also why the service runs one worker process.
- **One process.** Inference runs one call at a time behind a lock, and rate-limit counters live in that process. This is fine for one instance, and neither would carry over to several.
- **Before a class.** A teacher should open the site a minute or two beforehand if the server may be asleep.

11 Known limits and next steps

These limits are real and affect how Rollcall should be used today. The right-hand column shows what would address each one. Nothing there has been started.

Table 12 Known limits.

Limit	What it means	Next step
No liveness detection	A good photo may fool the camera. Teachers should watch the camera and correct the register by hand.	Add a liveness check, and test it against printed and on-screen photos.
No forgot-password email	There is no email service yet, so a forgotten password cannot be reset by the user.	Add an email provider and a reset flow with single-use links.
Free-tier cold start	The first request after idle can take about a minute. The site explains the wait, but the wait remains.	Move the API to an always-on instance.
In-memory rate limits, one process	Counters reset on restart, and the design does not extend to several instances. Recognition requests run one at a time.	Keep counters in a shared store, and run more processes if classes need parallel recognition.
Lighting and angle matter	Small faces are ignored, and poor light lowers scores. The app works best with one or two people in view at a time.	Measure in real rooms, then write guidance on camera placement.
Only synthetic faces in product testing	The API tests and browser flow use AI-generated faces. The LFW benchmark is the only measurement on photographs of real people. No real class has used it for a study.	Run a consented pilot with a real class, and record false accepts and rejects.
Fairness not measured	Accuracy has not been broken down across groups of people, and the overall LFW figures cannot show this.	Include this in any pilot, before relying on the register for decisions about students.

Taken together, this means the register is an assistant to the teacher, and the teacher remains responsible for it. Rollcall should not be the only evidence in any decision that affects a student until a real pilot has measured its error rates.

Appendix: local setup

This needs Docker, Python 3.11 and Node 22 or later. Commands run from the repository root.

Database

```
docker run -d --name rollcall-pg -e POSTGRES_PASSWORD=postgres -e POSTGRES_DB=rollcall \
  -p 5433:5432 postgres:17-alpine
```

Backend

```
cd backend
python3.11 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
mkdir -p models
curl -fL -o models/face_detection_yunet_2023mar.onnx \
  https://github.com/opencv/opencv_zoo/raw/main/models/face_detection_yunet/face_detection_yunet_2023mar.onnx
curl -fL -o models/face_recognition_sface_2021dec.onnx \

https://github.com/opencv/opencv_zoo/raw/main/models/face_recognition_sface/face_recognition_sface_2021dec.onnx
cp .env.example .env          # then set SECRET_KEY
python app.py                 # http://localhost:5000
```

Frontend

```
cd frontend
npm install
cp .env.example .env.local
npm run dev                   # http://localhost:3000
```

Tests

The API tests drop and recreate the `public` schema of their database, which by default is the same local database as above. Never point `TEST_DATABASE_URL` at data you want to keep.

```
cd backend && pip install pytest
FACE_DIR=<folder with at least 6 JPEGs, one distinct face each> pytest tests/test_api.py

cd frontend
FACE_DIR=<folder with p1.y4m p2.y4m p3.y4m room.y4m> node e2e/flow.mjs
```

The browser flow needs the site and API running, and the faces in its feeds must not already be enrolled, because the duplicate check would refuse them. The README has more detail, including how to rebuild this report and the GIFs.